Projet Informatique Individuel

Rendu Intermédiaire

GLSL Generator

Quentin Lopez

9 mars 2026

Table des matières

1 Introduction **1**

2 Travail réalisé **1**
2.1 Architecture technique 1
2.2 Interface et fonctionnalités principales 2
2.3 Blocs procéduraux implémentés 2

3 Avancement du projet **4**
3.1 Suite du projet 4
3.2 Difficultés rencontrées 4
3.3 Nouveau planning 4

1 Introduction

Ce projet vise à développer une application web capable de générer dynamiquement des shaders procéduraux en GLSL, avec prévisualisation 3D en temps réel. L'utilisateur peut sélectionner un matériau de base, modifier ses paramètres et observer immédiatement le rendu, tout en ayant accès au code du shader généré.

L'application utilise HTML/CSS pour l'interface, JavaScript pour la logique et GLSL pour les shaders. Le rendu 3D est assuré par la bibliothèque **Three.js** via WebGL. À ce stade, l'architecture générale est opérationnelle, l'interface fonctionnelle, et les premiers blocs procéduraux ont été développés.

2 Travail réalisé

2.1 Architecture technique

La page HTML charge le CSS et le fichier `main.js`, qui gère l'interface et la scène 3D. L'UI permet de sélectionner des shaders procéduraux, déclenchant le Shader Builder qui génère le code GLSL appliqué aux modèles 3D (GLTF) dans le fichier scene utilisant (Three.js) pour un rendu temps réel. Le fichier scene, gère également la camara, le rendu et le chargement des modèles.

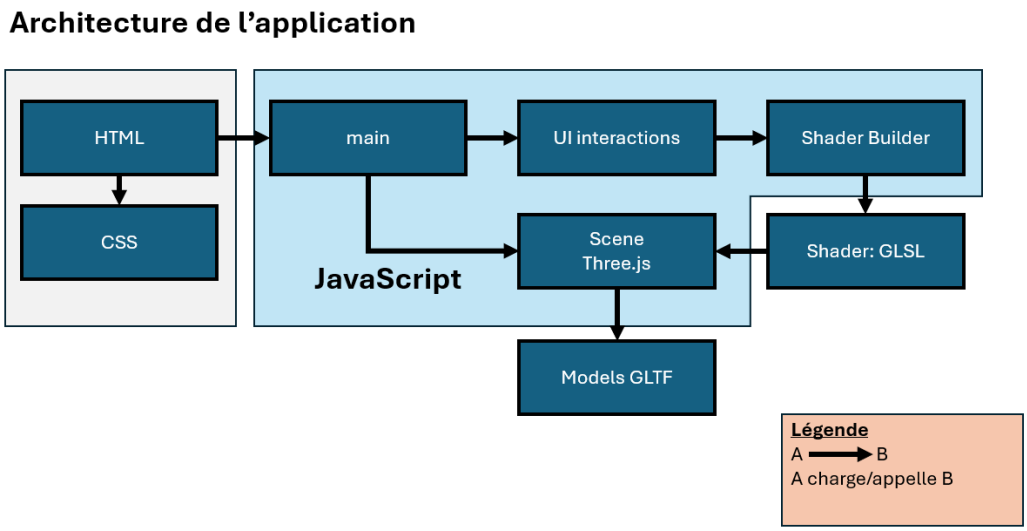

FIGURE 1 – Architecture de l'application

2.2 Interface et fonctionnalités principales

L'écran est organisé en quatre zones principales :

1. **Paramètres du matériau** : choix du matériau de base et options complémentaires.
2. **Prévisualisation 3D** : rendu temps réel du shader sur le modèle.
3. **Paramètres de la scène** : modification du background et sélection du modèle 3D (cube, sphère, torus ou fichiers GLTF : Suzanne, rock, cloth, gear). Le shader est appliqué automatiquement aux modèles importés.
4. **Code GLSL** : visualisation et téléchargement des vertex et fragment shaders.

Les principales fonctionnalités 3D incluent la navigation orbitale, le zoom et les vues prédéfinies.

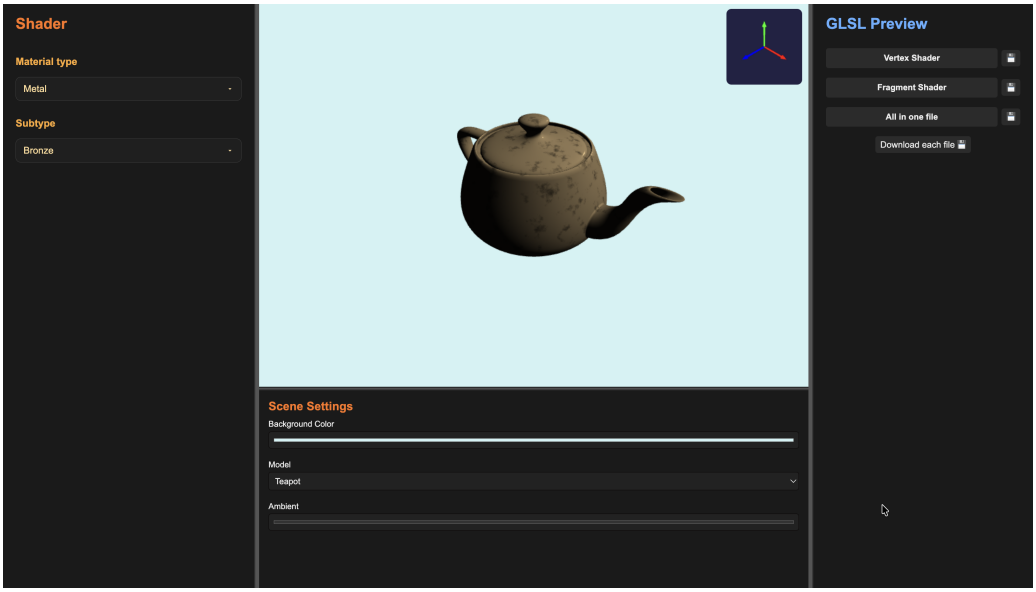

FIGURE 2 – Interface de l'application

2.3 Blocs procéduraux implémentés

Afin de générer des matériaux dynamiques, l'application repose sur un système de **blocs procéduraux** inspiré du fonctionnement des *shader nodes* dans **Blender**. Chaque bloc correspond à une unité de calcul indépendante capable de produire ou de modifier un motif appliqué à un matériau. L'objectif est de pouvoir combiner ces blocs afin de créer des shaders complexes, modulaires et réutilisables.

Chaque bloc possède ses propres paramètres et peut être connecté à différents attributs du matériau, notamment :

— la **couleur** (*color*),

— la **rugosité** (*roughness*), qui influence la réflexion et l'aspect brillant de la surface,

— le **bump ou normal**, qui permet de simuler un relief de surface sans modifier la géométrie.

Plusieurs blocs ont déjà été développés et constituent la base du système de génération procédurale. Dans l'architecture actuelle de l'application, ces blocs sont divisés en deux catégories :

— les **patterns**, qui génèrent des motifs procéduraux,

— les **opérateurs**, qui permettent de combiner ces motifs ou d'appliquer différentes transformations.

Une section de test a également été mise en place afin de visualiser individuellement chaque motif implémenté.

Patterns implémentés

- **Noise** : génère un bruit fractal produisant des variations aléatoires et naturelles sur la surface. Ce type de motif est particulièrement utile pour simuler des matériaux comme le bois, la pierre ou le métal usé.
- **Voronoi** : produit des motifs cellulaires pouvant rappeler des textures de pierre, de carrelage ou certaines structures organiques.
- **Wave** : génère des motifs ondulatoires suivant un axe donné. Ce bloc peut être utilisé pour créer des effets de bois, de tissus ou de métal ondulé.
- **Magic Texture** : crée des motifs colorés complexes et légèrement psychédéliques. Il est principalement utilisé pour tester les combinaisons de couleurs et le fonctionnement global du système procédural.

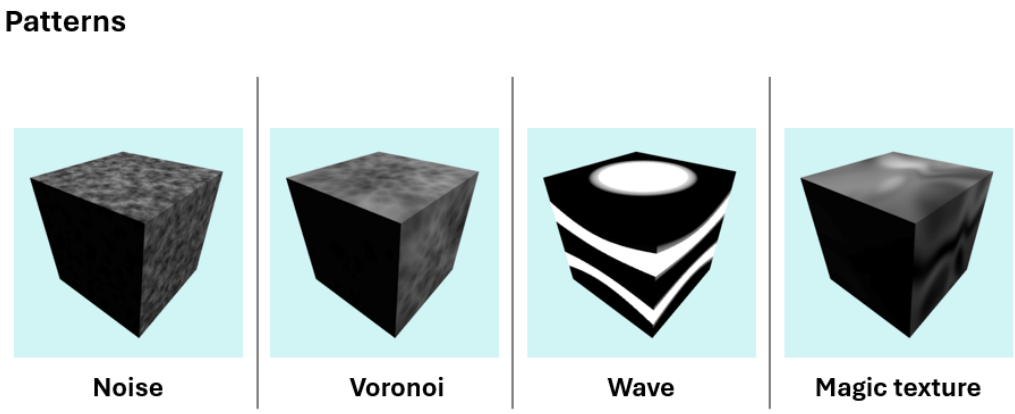

FIGURE 3 – Patterns

Opérateurs et blocs utilitaires

- **Mapping** : définit et transforme les coordonnées utilisées par les textures procédurales. Ce bloc sert généralement de point de départ pour la génération des motifs.
- **Mix** : permet de combiner plusieurs blocs entre eux selon différentes méthodes (addition, multiplication ou interpolation), afin de créer des motifs plus riches et variés.
- **ColorRamp** : permet de remapper les valeurs d'un motif vers un dégradé de couleurs. Ce bloc est souvent utilisé pour transformer un motif en texture colorée ou pour contrôler précisément les transitions entre différentes teintes.
- **BumpNode** : permet de transformer un motif en information de relief (*bump map*) afin de simuler des irrégularités de surface sans modifier la géométrie du modèle.
- **Connection** : gère la sortie finale du graphe et l'association des résultats avec les propriétés du matériau (couleur, rugosité, normal, etc.).

De cette manière, chaque matériau est représenté par un **graphe procédural** stocké dans un fichier séparé. Lorsqu'un matériau est sélectionné, le système charge le graphe correspondant et assemble automatiquement les différents blocs afin de générer le shader final.

À ce stade du projet, le système permet déjà de charger différents graphes et de basculer entre plusieurs matériaux.

3 Avancement du projet

3.1 Suite du projet

La suite du projet se déroulera en trois étapes principales :

— améliorer et ajuster les blocs procéduraux existants,

— créer de nouveaux matériaux procéduraux,

— permettre la modification de certains paramètres des matériaux directement depuis l'interface utilisateur.

3.2 Difficultés rencontrées

Par rapport au planning initial, l'implémentation des premiers matériaux procéduraux devait être plus avancée à ce stade. Un léger retard a toutefois été constaté, principalement dû à certaines difficultés techniques liées à **WebGL** et au langage **GLSL** :

— les erreurs WebGL sont parfois difficiles à diagnostiquer, se traduisant par exemple par un écran noir ou des messages d'erreur peu explicites ;

— GLSL est très sensible au typage et à la casse des identifiants, ce qui rend l'assemblage dynamique du code plus fragile et nécessite une attention particulière.

3.3 Nouveau planning

Le tableau suivant présente le nouveau planning prévu pour la suite du projet.

Période	**Tâches et objectifs**
Début mars	Mise en place du système de graphes pour les matériaux.
Fin mars	Ajout du paramétrage avancé des matériaux depuis l'interface utilisateur.
Point avant rendu final	**31 mars à 9h30**
Début avril	Finalisation du projet et correction des principaux bugs.
Mi-avril	Préparation de la soutenance et finalisation du rapport.
Soutenance finale	**21–22 avril (10 minutes par élève)**